CAMPOS IA

AULA 02

APOSTILA PRINCIPAL

Conceitos essenciais, explicações técnicas, exemplos e aplicações práticas

FUNDAMENTOS DE INFORMÁTICA
Hardware, software, dados, redes e segurança

SISTEMAS OPERACIONAIS
Funções, tipos, processos, memória e comandos

FERRAMENTAS DE PROGRAMAÇÃO
Linguagens, IDEs, Git, depuração e ferramentas modernas

Material didático de apoio • 2026

Apresentação

Esta apostila apresenta uma visão integrada de três áreas fundamentais para a formação técnica em tecnologia: os fundamentos de informática, o funcionamento dos sistemas operacionais e o conjunto de ferramentas utilizadas no desenvolvimento de software. O objetivo é compreender não apenas definições, mas também como os componentes se relacionam em situações reais.

Objetivos de aprendizagem

- Distinguir hardware, software, dados e informação e reconhecer como interagem.
- Identificar componentes de computadores e compreender noções de desempenho e armazenamento.
- Explicar a finalidade de um sistema operacional, seus processos, memória e sistemas de arquivos.
- Utilizar conceitos básicos de redes e reconhecer práticas essenciais de segurança digital.
- Compreender linguagens, ambientes de desenvolvimento, controle de versão e depuração.

1. Fundamentos de Informática

1.1 O que é informática?

Informática é a área que estuda o tratamento automatizado da informação por meio de computadores e sistemas digitais. Um sistema computacional recebe dados, processa-os segundo instruções, armazena resultados e os apresenta aos usuários ou a outros sistemas. O ciclo básico pode ser resumido em entrada, processamento, saída e armazenamento.

1.2 Dados, informação e conhecimento

Dado é um registro bruto, como o número 28, uma data ou uma leitura de sensor. **Informação** surge quando os dados são organizados e contextualizados: “28 alunos participaram da atividade”. **Conhecimento** envolve interpretar a informação e utilizá-la para tomar decisões. A qualidade da informação depende de precisão, atualidade, integridade e relevância.

1.3 Hardware: componentes físicos

Hardware compreende as partes físicas do computador. A placa-mãe interliga os componentes; a CPU executa instruções; a memória RAM mantém temporariamente dados e programas em uso; o armazenamento SSD ou HD conserva arquivos mesmo após o desligamento. A fonte fornece energia, enquanto dispositivos de entrada e saída permitem a interação.

- CPU: contém unidades de controle e execução; seu desempenho depende de arquitetura, frequência, núcleos, cache e carga de trabalho.
- RAM: memória volátil, rápida e utilizada por programas ativos. Mais RAM pode reduzir o uso de memória virtual, mas não substitui um processador adequado.
- SSD/HD: armazenamento não volátil. SSDs usam memória flash e geralmente oferecem menor latência que HDs mecânicos.
- GPU: processa operações gráficas e, em modelos apropriados, tarefas paralelas de computação científica e inteligência artificial.
- Periféricos: teclado, mouse, câmera, microfone, monitor, impressora e dispositivos externos.

1.4 Software: programas e sistemas

Software é o conjunto de instruções e dados que orienta o hardware. O software de sistema administra recursos e oferece serviços básicos, como sistemas operacionais e drivers. O software aplicativo atende necessidades do

usuário, como navegadores, editores, planilhas e sistemas acadêmicos. Ferramentas de desenvolvimento auxiliam na criação, teste e manutenção de programas.

1.5 Representação digital

Computadores representam dados em bits, que assumem os valores 0 e 1. Um conjunto de oito bits forma um byte. Textos são codificados por padrões como Unicode; imagens podem ser representadas por pixels; áudio é convertido em amostras digitais; vídeos combinam quadros e áudio. Unidades comuns incluem KB, MB, GB e TB. É importante distinguir capacidade de armazenamento de velocidade de transferência.

1.6 Redes de computadores

Uma rede conecta dispositivos para compartilhar dados e serviços. Em uma rede local (LAN), os equipamentos estão em uma área limitada, como laboratório ou residência. A internet é uma rede mundial de redes. Equipamentos como switches conectam dispositivos em uma LAN e roteadores encaminham pacotes entre redes. Wi■Fi permite conexão sem fio.

Protocolos definem regras de comunicação. IP endereça dispositivos; TCP oferece transporte confiável e ordenado; UDP prioriza simplicidade e baixa latência em aplicações compatíveis; DNS traduz nomes de domínio em endereços IP; HTTP e HTTPS são usados na comunicação web, sendo HTTPS protegido por TLS. A largura de banda representa capacidade de transmissão, enquanto latência é o tempo de resposta.

1.7 Segurança da informação

Segurança da informação busca preservar confidencialidade, integridade e disponibilidade. Confidencialidade limita o acesso a pessoas autorizadas; integridade protege contra alterações indevidas; disponibilidade garante acesso quando necessário. Ameaças incluem phishing, malware, engenharia social, vazamento de credenciais e falhas de configuração.

- Use senhas longas, únicas e gerenciadas por um gerenciador confiável.
- Ative autenticação multifator sempre que possível.
- Mantenha sistema, navegador e aplicativos atualizados.
- Faça cópias de segurança e teste a restauração.
- Desconfie de links, anexos e pedidos urgentes de dados ou pagamentos.
- Conceda somente as permissões necessárias e bloqueie a tela ao se afastar.

2. Sistemas Operacionais

2.1 Conceito e finalidade

Sistema operacional (SO) é o software básico que gerencia o hardware e oferece uma camada de serviços para aplicativos e usuários. Ele coordena processador, memória, dispositivos, arquivos, usuários e mecanismos de proteção. Sem o SO, cada programa precisaria controlar diretamente os detalhes do hardware, tornando o uso do computador muito mais complexo.

2.2 Tipos de sistemas operacionais

Há sistemas para desktops e notebooks, servidores, dispositivos móveis, sistemas embarcados e equipamentos de tempo real. Windows, distribuições Linux e macOS são exemplos de sistemas para computadores pessoais; Android e iOS são voltados a dispositivos móveis. Sistemas embarcados aparecem em roteadores, automóveis, sensores e equipamentos industriais.

2.3 Processos e threads

Um programa é um conjunto de instruções armazenado; um processo é uma instância em execução desse programa, com recursos e espaço de memória próprios. Uma thread é uma sequência de execução dentro de um processo. Processos podem estar em estados como novo, pronto, em execução, bloqueado e encerrado. O escalonador decide qual processo ou thread recebe tempo de CPU.

A multitarefa permite que vários processos avancem aparentemente ao mesmo tempo. Em processadores com múltiplos núcleos, tarefas podem executar em paralelo. Trocas de contexto permitem alternar a execução, mas têm custo. Concorrência exige atenção a condições de corrida, deadlocks e sincronização.

2.4 Gerenciamento de memória

O SO administra a memória física e virtual. A memória virtual oferece a cada processo um espaço de endereçamento lógico e permite utilizar armazenamento como apoio quando a RAM é insuficiente. Paginação divide a memória em blocos; o sistema movimenta páginas conforme a necessidade. Uso intenso de paginação pode reduzir o desempenho, pois armazenamento é mais lento que RAM.

2.5 Arquivos, diretórios e permissões

O sistema de arquivos organiza dados em arquivos e diretórios. Cada arquivo possui nome, tipo, tamanho, metadados e permissões. Caminhos podem ser absolutos ou relativos. Permissões definem quem pode ler, gravar ou executar. Organização consistente e cópias de segurança facilitam manutenção e recuperação.

2.6 Interface gráfica e linha de comando

A interface gráfica oferece janelas, menus e ícones. A linha de comando permite executar instruções textuais, automatizar tarefas e administrar sistemas remotamente. Comandos variam entre sistemas e shells; antes de executar comandos destrutivos, confirme o diretório e entenda seus parâmetros.

2.7 Comandos introdutórios

Objetivo	Windows (PowerShell)	Linux (Bash)
Listar arquivos	Get-ChildItem	ls
Mostrar diretório atual	Get-Location	pwd
Mudar diretório	Set-Location pasta	cd pasta
Criar diretório	New-Item -ItemType Directory nome	mkdir nome
Copiar arquivo	Copy-Item origem destino	cp origem destino
Exibir conteúdo	Get-Content arquivo.txt	cat arquivo.txt

3. Ferramentas de Programação

3.1 Linguagens de programação

Uma linguagem de programação define regras de sintaxe e semântica para expressar algoritmos. Linguagens compiladas são traduzidas previamente ou durante a execução para código de máquina ou representação intermediária. Linguagens interpretadas são executadas por um interpretador ou máquina virtual. Na prática, muitas plataformas combinam compilação, interpretação e otimização.

Python é conhecida por sua legibilidade e ecossistema científico; JavaScript é central no desenvolvimento web; Java e C# são utilizadas em aplicações corporativas; C e C++ oferecem controle de baixo nível e alto desempenho; SQL é utilizada para consultar e manipular bancos de dados. A escolha depende do problema, ambiente, equipe, requisitos e manutenção.

3.2 Algoritmos e estrutura de um programa

Algoritmo é uma sequência finita e organizada de passos para resolver um problema. Um programa transforma um algoritmo em instruções executáveis. Variáveis armazenam valores; tipos definem operações válidas;

estruturas condicionais permitem decisões; laços repetem operações; funções organizam código reutilizável.

Exemplo em Python:

```
nota = 8.0
if nota >= 7.0:
    print('Aprovado')
else:
    print('Em recuperação')
```

3.3 IDEs e editores

Uma IDE (ambiente integrado de desenvolvimento) reúne editor, execução, depuração, integração com compiladores e outras ferramentas. Editores como Visual Studio Code podem ser ampliados por extensões. IDEs completas podem oferecer gerenciamento de projetos, refatoração, análise estática e integração com testes. A ferramenta deve ser configurada de acordo com a linguagem e o projeto.

3.4 Git e controle de versão

Git registra alterações em arquivos ao longo do tempo. Um repositório contém o histórico; commits representam conjuntos de alterações identificados por mensagens; branches permitem desenvolver funcionalidades isoladamente; merge integra linhas de trabalho. Plataformas como GitHub e GitLab hospedam repositórios e apoiam colaboração, revisão e automação.

- git init: inicia um repositório local.
- git status: mostra arquivos modificados e estado do repositório.
- git add: prepara alterações para o próximo commit.
- git commit -m: registra as alterações preparadas.
- git log: consulta o histórico.
- git branch e git switch: gerenciam e alternam branches.
- git pull e git push: sincronizam alterações com um repositório remoto.

3.5 Depuração e testes

Depurar é investigar a causa de um comportamento incorreto. Um depurador permite pausar a execução, usar breakpoints, avançar linha a linha e inspecionar variáveis. Logs registram eventos; testes verificam resultados esperados. Testes unitários avaliam pequenas unidades; testes de integração verificam interação entre componentes; testes de sistema avaliam o conjunto.

3.6 Boas práticas de desenvolvimento

Código legível reduz erros e facilita manutenção. Use nomes claros, funções com responsabilidades definidas, comentários que expliquem decisões relevantes, formatação consistente e controle de versão. Revise dependências, proteja segredos fora do código-fonte e documente como instalar, executar e testar o projeto.

3.7 Ferramentas modernas e fluxo de trabalho

Projetos atuais podem utilizar gerenciadores de pacotes, ambientes virtuais, containers, integração contínua, análise estática, ferramentas de API e plataformas de hospedagem. Ferramentas de inteligência artificial podem auxiliar na explicação de código e geração de rascunhos, mas resultados devem ser revisados, testados e avaliados quanto à segurança e licenças.

4. Integração dos três assuntos

Imagine um laboratório que desenvolve um sistema de controle de equipamentos. O hardware fornece processamento e armazenamento; a rede conecta computadores e servidor; o sistema operacional gerencia

processos, arquivos e permissões; a linguagem de programação implementa as funcionalidades; a IDE apoia a escrita e depuração; Git registra a evolução. Segurança atravessa todas as etapas: credenciais protegidas, atualizações, backups e controle de acesso.

5. Síntese e revisão

Informática envolve o processamento de dados por sistemas computacionais. Hardware executa e armazena; software orienta o funcionamento. Redes permitem comunicação e segurança protege recursos e informações. O sistema operacional administra processos, memória, arquivos e dispositivos. Ferramentas de programação apoiam a construção, teste, colaboração e manutenção de soluções digitais.

Glossário

- Bit/byte: unidade básica de informação digital; oito bits formam um byte.
- CPU: unidade central de processamento.
- RAM: memória principal volátil.
- Protocolo: conjunto de regras de comunicação.
- Processo: programa em execução.
- Thread: fluxo de execução dentro de um processo.
- IDE: ambiente integrado de desenvolvimento.
- Commit: registro de alterações no Git.
- Breakpoint: ponto de interrupção usado na depuração.
- Backup: cópia destinada à recuperação de dados.

Referências para aprofundamento

- TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. Pearson.
- STALLINGS, William. Arquitetura e Organização de Computadores. Pearson.
- KERNIGHAN, Brian W.; RITCHIE, Dennis M. The C Programming Language. Prentice Hall.
- Documentação oficial do Python: docs.python.org/pt-br/
- Documentação oficial do Git: git-scm.com/doc.
- Documentação do Microsoft Learn: learn.microsoft.com.

Este material é didático e introdutório. Comandos, recursos e interfaces podem variar conforme versão do sistema operacional e configuração do ambiente.